Name: Shreya Aggarwal

CU24250082

Roll No: 58

Section: C

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
video_path = "/content/optical_flow.mp4"

print("Video path:", video_path)
```

```
Video path: /content/optical_flow.mp4
```

```
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open the video.")
else:
    print("Video opened successfully!")

total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print("Total Frames:", total_frames)
print("FPS:", fps)
print("Width:", width)
print("Height:", height)
```

```
Video opened successfully!
Total Frames: 90
FPS: 30.0
Width: 640
Height: 360
```

```
cap = cv2.VideoCapture(video_path)

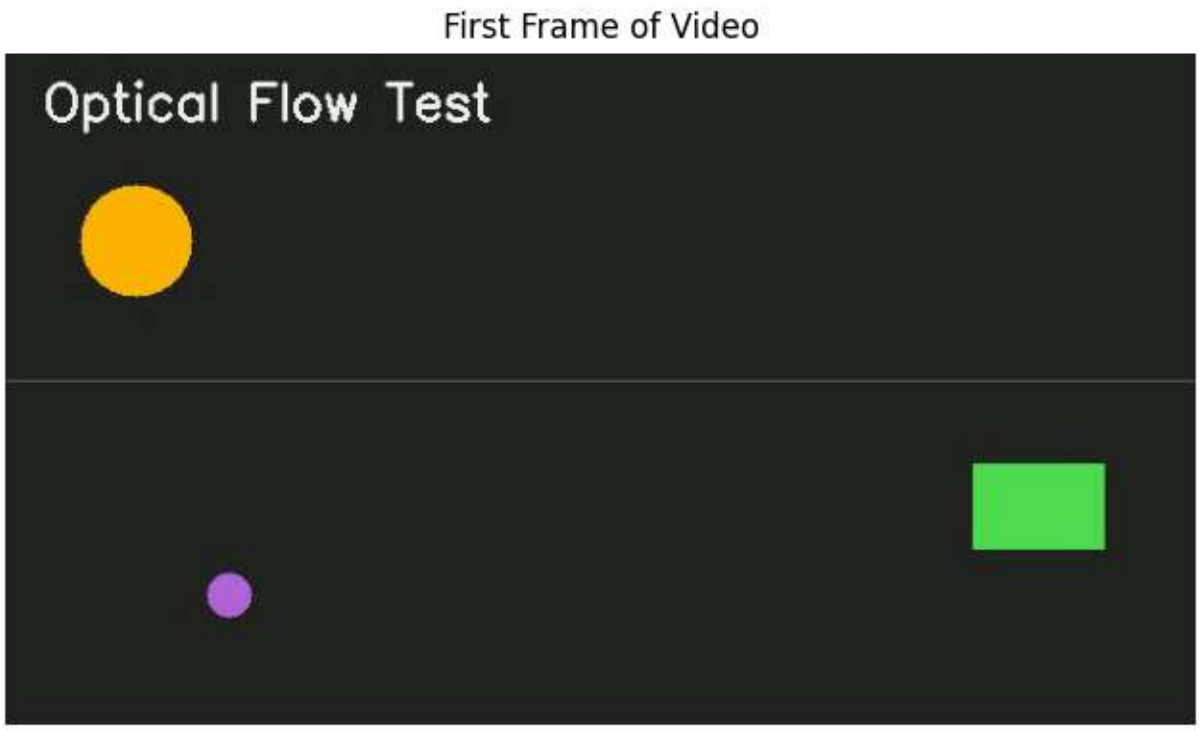
ret, frame = cap.read()

if ret:
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(8, 5))
```

```
    plt.imshow(frame_rgb)
    plt.title("First Frame of Video")
    plt.axis("off")
    plt.show()
else:
    print("Could not read the first frame.")

cap.release()
```




```
cap = cv2.VideoCapture(video_path)

ret, frame1 = cap.read()
ret, frame2 = cap.read()

if ret:
    gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    print("Two consecutive frames loaded successfully.")
else:
    print("Could not read the video frames.")

cap.release()
```

```
Two consecutive frames loaded successfully.
```

```
cap = cv2.VideoCapture(video_path)

ret, first_frame = cap.read()
```

```
gray_first = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)

feature_points = cv2.goodFeaturesToTrack(
    gray_first,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

print("Number of feature points detected:", len(feature_points))

cap.release()
```

```
Number of feature points detected: 31
```

```
display_frame = first_frame.copy()

for point in feature_points:
    x, y = point.ravel()
    cv2.circle(display_frame, (int(x), int(y)), 5, (0, 255, 0), -1)

plt.figure(figsize=(8, 5))
plt.imshow(cv2.cvtColor(display_frame, cv2.COLOR_BGR2RGB))
plt.title("Selected Feature Points")
plt.axis("off")
plt.show()
```



```
cap = cv2.VideoCapture(video_path)

ret, old_frame = cap.read()

old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

ret, new_frame = cap.read()

new_gray = cv2.cvtColor(new_frame, cv2.COLOR_BGR2GRAY)

p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray,
    new_gray,
    p0,
    None
)

good_new = p1[status == 1]
good_old = p0[status == 1]

print("Successfully tracked points:", len(good_new))

cap.release()
```

```
Successfully tracked points: 31
```

```
lk_output = new_frame.copy()

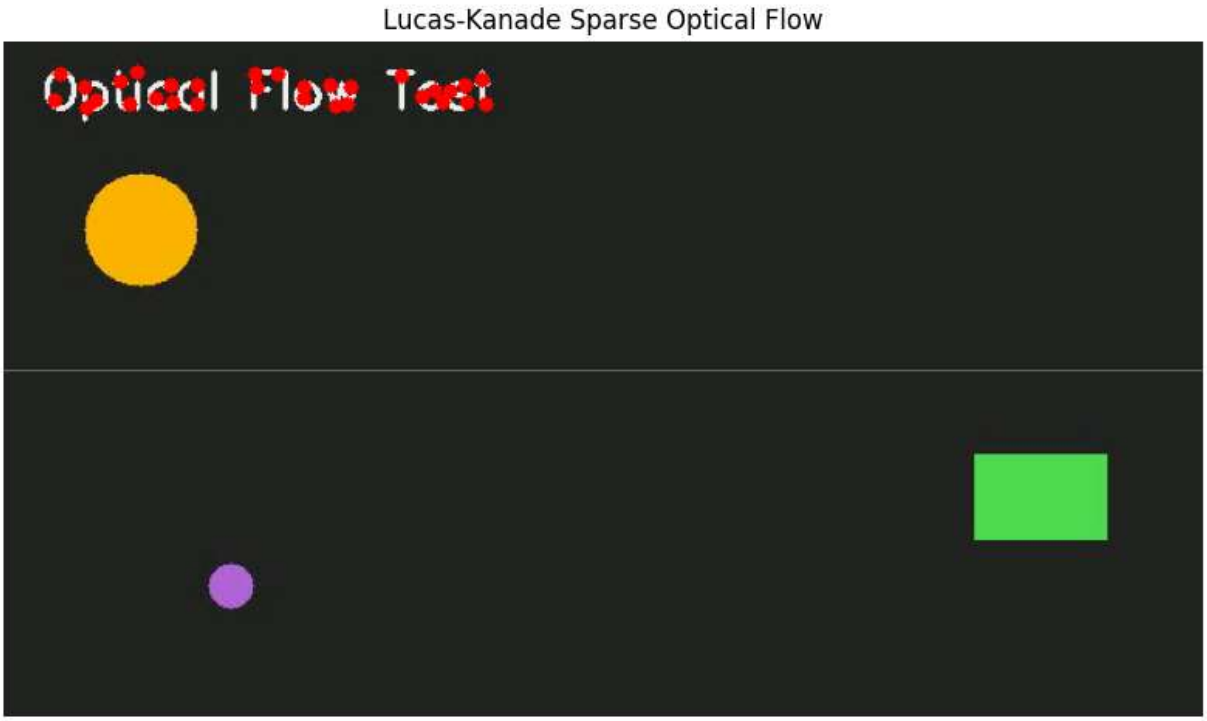
for new, old in zip(good_new, good_old):
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    cv2.arrowedLine(
        lk_output,
        (int(x_old), int(y_old)),
        (int(x_new), int(y_new)),
        (0, 255, 0),
        2,
        tipLength=0.3
    )

    cv2.circle(
        lk_output,
```

```
            (int(x_new), int(y_new)),
            4,
            (0, 0, 255),
            -1
        )

    plt.figure(figsize=(10, 6))
    plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
    plt.title("Lucas-Kanade Sparse Optical Flow")
    plt.axis("off")
    plt.show()
```




```
    cap = cv2.VideoCapture(video_path)

    ret, frame1 = cap.read()
    ret, frame2 = cap.read()

    gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    cap.release()

    print("Frames ready for Farneback Optical Flow.")
```

```
Frames ready for Farneback Optical Flow.
```

```
flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

print("Farneback Dense Optical Flow calculated successfully.")
```

```
Farneback Dense Optical Flow calculated successfully.
```

```
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

hsv = np.zeros_like(frame1)

hsv[..., 1] = 255

hsv[..., 0] = angle * 180 / np.pi / 2

hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

dense_flow_output = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)

plt.figure(figsize=(10, 6))
plt.imshow(dense_flow_output)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()
```

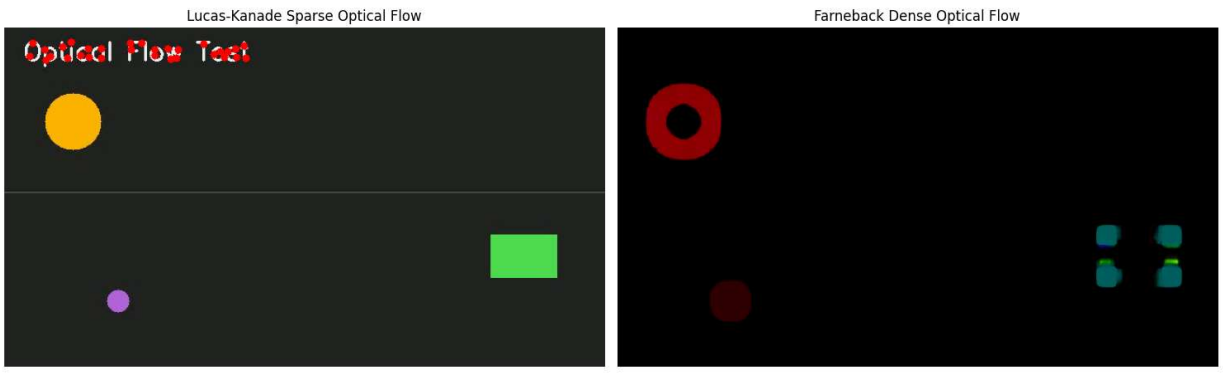



```
plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(dense_flow_output)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")

plt.tight_layout()
plt.show()
```



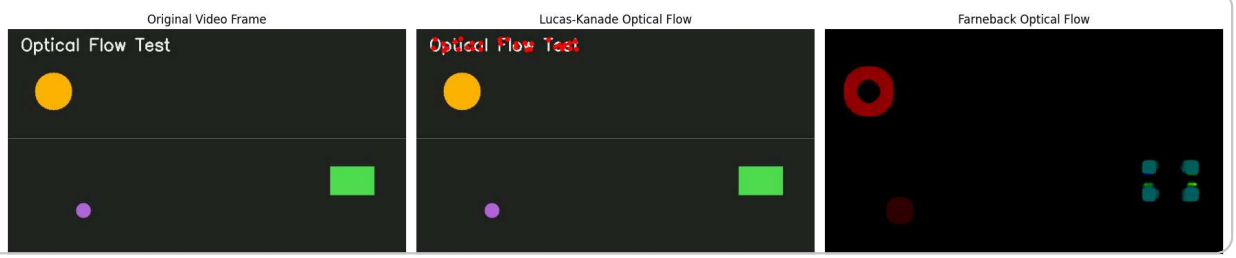

```
plt.figure(figsize=(18, 5))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB))
plt.title("Original Video Frame")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Optical Flow")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(dense_flow_output)
plt.title("Farneback Optical Flow")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
cv2.imwrite(
    "/content/lucas_kanade_output.jpg",
    lk_output
)

cv2.imwrite(
    "/content/farneback_output.jpg",
    cv2.cvtColor(dense_flow_output, cv2.COLOR_RGB2BGR)
)

print("Optical flow results saved successfully.")
```

```
Optical flow results saved successfully.
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.